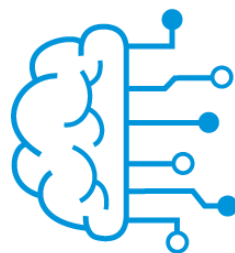


Informe Técnico BDIA

Alumno	Rubiolo Pedro Ignacio
Título	Sistema de Gestión Integral para StartUps
Fecha	26/08/2026



Laboratorio
de **Sistemas
Embebidos**

1. Análisis del caso de uso	3
2. Relevamiento de datos necesarios	4
3. Modelo conceptual	5
4. Modelo lógico	6
5. Normalización, desnormalización y decisiones de diseño	8
6. Selección tecnológica	10
7. Implementación mínima	11
8. Datos de ejemplo utilizados	12
9. Consultas representativas	13
10. Propuesta para datos semiestructurados, no estructurados o vectoriales.	14
11. Arquitectura de datos	15
12. Seguridad, permisos y aislamiento	17
13. Escalabilidad y rendimiento	18
14. Conclusiones	19

1. Análisis del caso de uso

El objetivo de este proyecto es diseñar un sistema de uso interno, destinado a StartUps cuya unidad de negocio se basa en el diseño y fabricación de proyectos. Este sistema debe permitir:

- Llevar registro de los stakeholders (clientes, empleados, proveedores).
- Llevar registro de compras y ventas
- Llevar registro de proyectos, incluyendo:
 - Documentación, con control de versiones
 - Archivos, con control de versiones
 - Componentes
 - Implicados
- Llevar control de stock

Adicionalmente, este sistema podrá incorporar un agente de IA que permita al usuario hacer consultas y obtener respuestas en base a la información disponible del sistema. Por ejemplo

- ¿Cuál es el stock disponible de ...?
- ¿Qué componentes hace falta conseguir para construir el proyecto ...?
- ¿Haz un resumen del proyecto ...?

Este sistema soluciona el problema de no tener la información centralizada y organizada, lo que complica la trazabilidad de todos los aspectos del negocio, así como brindar la posibilidad de hacer consultas en lenguaje natural y obtener respuestas basadas en los datos propios.

En primera instancia, este sistema solo será utilizable por aquellos que pertenecen a la organización, presentando control de acceso en función de su participación en el proyecto y rol asignado.

Se deberá tener precaución a la hora de manejar el versionado de los documentos ya que se presenta el riesgo de perder información así como también brindar versiones obsoletas. Por otro lado, se debe tener cuidado con los componentes ya que son susceptibles a ser duplicados por accidente.

Para lograr la adopción del sistema se deberá conveniar con los interesados para conocer el flujo de operaciones y adaptar el sistema a ellas y no viceversa. Además, se debe procurar que la interacción con el mismo sea simple.

2. Relevamiento de datos necesarios

Se identifica que se deberán almacenar los siguientes datos:

Datos Estructurados:

- Información de clientes
- Información de proveedores
- Información de empleados
- Metadatos de partes
- Metadatos de documentos
- Metadatos de archivos

Datos Semiestructurados:

- Descripción del proyecto
- Descripción de parte

Datos no estructurados:

- Planos
- Documentos
- Modelos 3D
- Archivos

Datos Operacionales:

- Registro de egresos
- Registro de ingresos
- Stock

Datos de Auditoría:

- Versiones de documentos
- Versiones de archivos
- Movimientos de Stock

Para la demostración se generarán datos falsos para completar las distintas tablas y validar el funcionamiento del sistema.

3. Modelo conceptual

El modelo conceptual nos permite entender un poco más a alto nivel cuáles son las entidades vinculadas y las relaciones entre ellas. En este proyecto podemos decir que hay 4 entidades principales que dan sentido al proyecto. Estas son:

- **Personas:** Cualquier ente físico o jurídico que tenga vinculación con las operaciones de la empresa.
- **Proyectos:** Es la unidad de negocio de la empresa.
- **Documentos:** Es la información que la empresa genera tanto a nivel proyecto como a nivel organizativo
- **Partes:** Son aquellos componentes físicos que componen un proyecto.

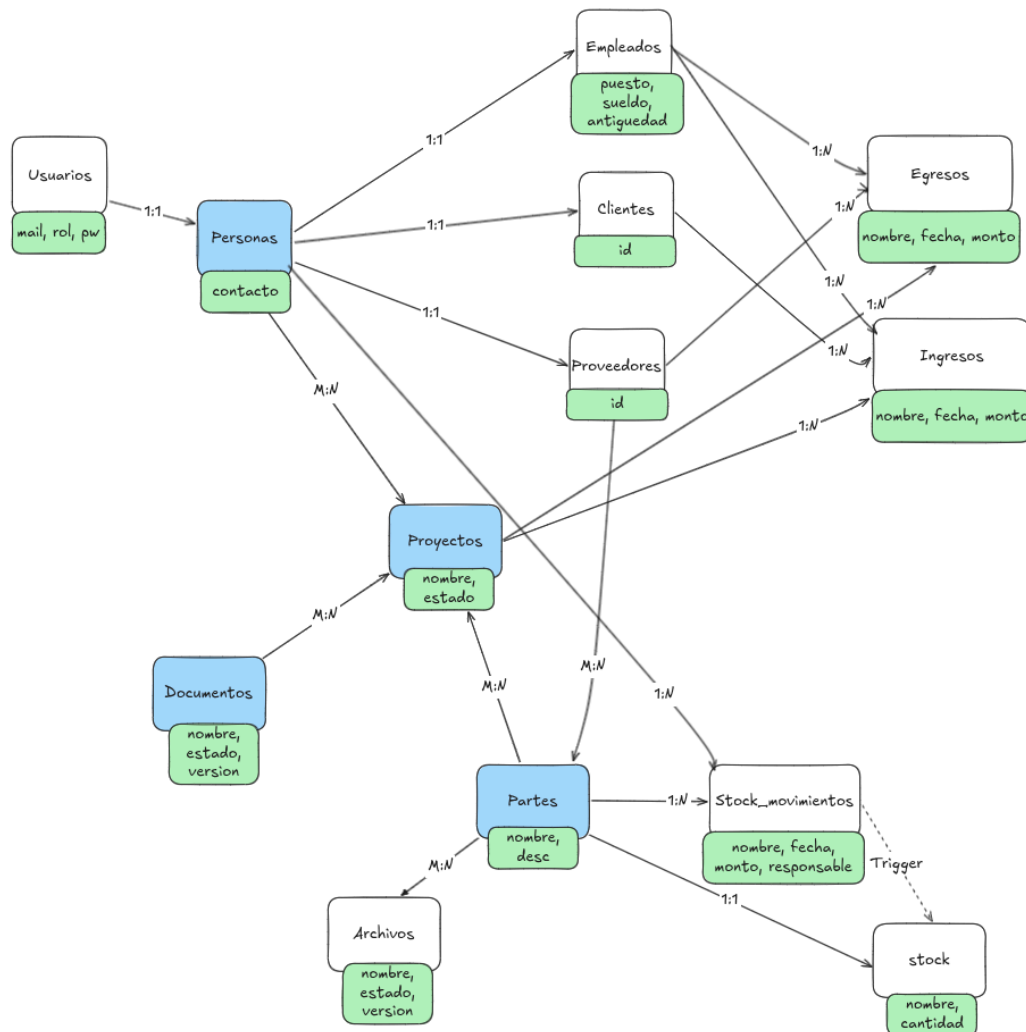


Figura 1 - Modelo Conceptual

En el diagrama de la **fig. 1** podemos observar la idea esencial de que un proyecto está compuesto por distintos interesados y que se da forma a través de los documentos y se concreta entregando un cúmulo de partes.

4. Modelo lógico

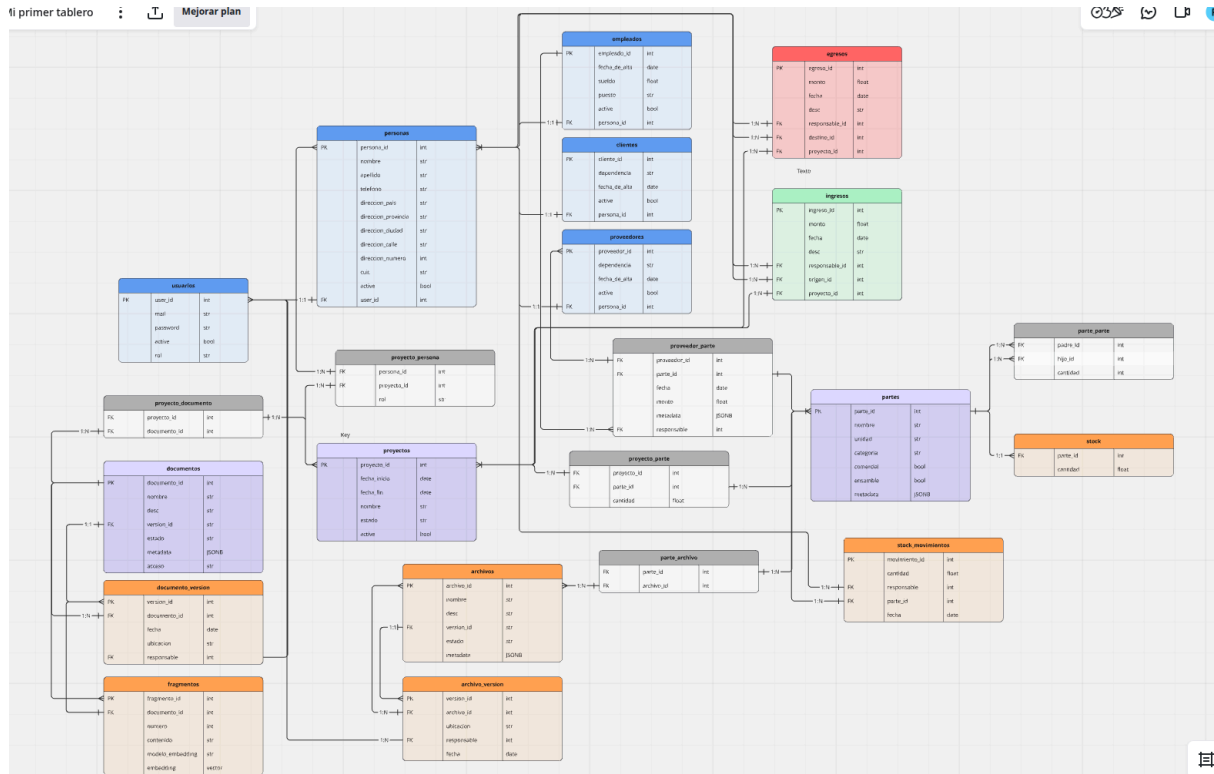


Figura 2 - Modelo Lógico

A la hora de hacer el modelo lógico, se toman distintas decisiones pensando en la compatibilidad a futuro y los patrones de acceso. Se enumeran los siguientes criterios adoptados.

- Se crea la entidad **personas**, que contiene información para identificar a una única persona real o jurídica. No se le asigna un rol directamente, sino que se lo corresponde a una de las tablas de **clientes**, **proveedores** o **empleados**. La motivación de esto es resolver la ambigüedad entre una persona que puede ser proveedor y cliente.
- Se crea la entidad **usuarios** que define las credenciales y niveles de acceso y se liga a una única persona real/jurídica. Esto permite la idea de tener una cuenta por persona y poder a futuro brindar acceso a personas no pertenecientes a la organización asignando un nivel de acceso.
- Se crean las entidades de **ingresos** y **egresos** en pos de una unificada con la motivación de poder establecer esquemas distintos para cada caso.
- Se crea la entidad **proyectos**, con el fin de poder asociar información a este. La idea es relacionar personas implicadas, gastos producidos, ingresos producidos, partes que utiliza y documentos relacionados.

- Se crea la entidad **partes**, pensada como un registro de partes que componen a un proyecto, pudiendo ser tanto comerciales como personalizadas. Una decisión fue la de dejar las partes como una entidad global, entendiéndose por oposición a una entidad que dependa del proyecto en particular. Esto último se infunde en que permitirá tener un control de stock más simple a la vez que se podrán replicar partes en distintos proyectos sin necesidad de crear nuevas. Estas partes pueden tener asociados uno o varios archivos como planos, especificaciones técnicas, etc. También se permite asociar las partes con los proveedores para tener registro de dónde se pueden conseguir y una entidad **partes_relacion** que permite manejar composiciones de ensambles.
- Se crea la entidad de **stock** con el fin de poder monitorear las existencias de partes que hay actualmente. Se decidió que no sea modificada directamente, si no que sea **stock_movimientos** la entidad empleada para ello a los fines de tener trazabilidad.
- Se crea la entidad **documentos**, entendiéndose como aquellos archivos de texto escrito que pueden o no estar vinculados a uno o más proyectos. Estos pueden ser normativas, planificaciones, reclamos, etc., y la idea es poder llevar un registro de versiones de los mismos mediante la tabla **documentos_versiones**, así como también representarlos de forma vectorial para su uso en sistemas RAG mediante la entidad **fragmentos**.

Se sugiere consultar el diagrama a través de la herramienta con la que fue creado, en el siguiente [enlace](#).

5. Normalización, desnormalización y decisiones de diseño

En el sistema propuesto, se usa mucha normalización, con el fin de tener mejor control sobre datos duplicados, así como también sobre operaciones de borrado.

En particular podemos destacar los siguientes ejemplos:

- Separar usuarios de personas.
- Separar personas de clientes, proveedores y empleados.
- Separar partes y stock
- Separar dirección de persona en varios campos
- Separar documentos de fragmentos
- Definir ensambles mediante parte_parte

Un punto que se decidió dejar normalizado, pero bien podría haber sido desnormalizado, es el que compete a la tabla de fragmentos. Si se hubiese optado por incorporar el tipo de acceso del documento al cual pertenece, así como sus implicados, se hubiese ahorrado hacer múltiples joins al evaluar la política de RLS. La motivación para no desnormalizar fue que la cantidad de operaciones es muy baja y esta mejora en performance no sería tan provechosa como lo es el poder mantener información no duplicada, qué es más fácil de gestionar.

En cuanto a las operaciones de borrado, se establecieron las siguientes condiciones:

- Si una persona se borra del sistema, se la elimina de clientes, proveedores o empleados, proyectos. De todos modos, se prevé su eliminación a través del flag active. Si la persona registró un movimiento de stock o dinero, se rechaza el borrado para evitar pérdidas de trazabilidad.
- Si se borra un proyecto, se eliminan todas las relaciones establecidas en proyecto_persona, proyecto_parte, proyecto_documento. Si el proyecto tiene asociado algún movimiento, se bloquea el borrado para mantener la trazabilidad.
- Si se borra una parte, se eliminan las relaciones establecidas en parte_parte, parte_proyecto, parte_archivo, proveedor_parte
- Si se borra un archivo, se eliminan las relaciones establecidas en parte_archivo así como todo el registro de versiones del mismo.
- Si se borra un documento, se eliminan las relaciones establecidas en proyecto_documento, así como todas sus versiones y fragmentos asociados.

Volviendo a la propuesta original, las decisiones de diseño permiten acceder a los datos de la forma esperada:

- **Llevar registro de los stakeholders:** Se logra a través de consultar las tablas de clientes, proveedores y empleados.
- **Llevar registro de compras y ventas:** Se logra a través de consultar las tablas de ingresos y egresos.
- **Llevar registro de proyectos:** Se logra consultando información propia del proyecto en la tabla proyectos, así como sus relaciones mediante las tablas proyecto_documento, proyecto_persona, proyecto_parte, ingresos y egresos.

- **Llevar control de stock:** Se logra consultando directamente la tabla stock y su historial en la tabla stock_movimientos.
- **Llevar control de versiones:** Se logra mediante las tablas documento_version y archivo_version, permitiendo mantener distintas versiones en caso de que se necesite revisar cambios.
- **Consultas de documentación:** Se logran mediante la tabla de fragmentos, que permite ser consultada de forma vectorial.

6. Selección tecnológica

En este caso de uso se propone utilizar **postgres** acompañado de un data lake con **minio**.

La motivación principal de usar **postgres** radica en su versatilidad para representar múltiples tipos de datos en una sola base de datos. En la aplicación desarrollada se observan:

- **Datos estructurados** que son fácilmente representados mediante la base de datos relacional.
- **Datos no estructurados** en baja cantidad, previstos para dar flexibilidad a la hora de registrar documentos y partes que fueron solventados mediante JSONB permitiendo esta flexibilidad, así como la asociación con la base de datos relacional.
- **Datos vectoriales** para almacenar los embeddings de los documentos, que en este caso la extensión **pgvector** provee soporte de los mismos, con la ventaja de además mantener las asociaciones relacionales.

Debido a que en primera instancia se pensó como una herramienta interna, el volumen de datos no debería crecer de forma tal que se justifique alguna alternativa columnar o distribuida.

Por otro lado, se necesitó acompañarla con **minio** para poder almacenar los documentos y los archivos, asociando las **URIS** de los mismos en los atributos correspondientes de las tablas de **postgres**.

7. Implementación mínima

Para la implementación mínima se incluye:

- **Servicios** de minio, postgres, pgadmin y fastAPI
- **Creación** de todas las tablas presentes en modelo lógico, incluyendo relaciones, tipos de datos y restricciones.
- **Front-end** para probar funcionalidades básicas.
- **Endpoints** en fastAPI para interactuar con la base de datos desde el front end
- Implementación mínima para **consultas en lenguaje natural**.
- Implementación mínima para consultas por **búsqueda vectorial** (No incluye documentos fragmentados con sus embeddings).
- **Roles** en postgres.

En esta implementación no se incluye:

- **Fragmentación** y embeddings de documentos.
- Implementación de **RLS**

8. Datos de ejemplo utilizados

Se pobló la base de datos con información de clientes, empleados y proveedores ficticios mediante la carga de archivos .csv y cargas manuales mediante la interfaz.

Por otro lado se crea un proyecto con la siguiente estructura:

- **Nombre:** Mecanismo de elevación L21
- **Implicados:**
 - Rubiolo Pedro Ignacio, Diseño Mecatrónico
 - Pablo Perez, Relevamiento en planta
 - Rodolfo Arruabarrena, Seguridad e Higiene
 - Cristian Pavon, Representante Empresa SRL
- **Partes:**
 - Mecanismo de elevación L21 **x2**
 - Pinon 30L25 **x1**
 - **CAD** Pinon 30L25
 - Eje p/ mecanismo de elevación L21 **x1**
 - **CAD** Eje p/ mecanismo de elevación
 - Portarodamiento M2013 **x2**
 - **CAD** Portarodamiento M2013
 - **Datasheet** Portarodamiento M2013
 - Cadena 3PDL **x1**
 - **Datasheet** Cadena 3PDL
- **Documentos:**
 - Requisitos técnicos mecanismo de elevación L21
 - Normas para contratistas Empresa SRL

Se crean además versiones adicionales de las partes, se adjuntan cotizaciones y se registran movimientos de stock y dinero.

9. Consultas representativas

Se propone responder las siguientes consultas:

1. ¿Cuáles son los proyectos activos?
2. ¿Dónde se encuentra la última versión del archivo x?
3. ¿Qué componentes (desglosados) se requieren para llevar a cabo el proyecto x?
4. ¿Qué componentes faltan para llevar a cabo el proyecto x?
5. ¿Dónde puedo comprar la parte x?
6. ¿Qué archivos asociados tiene la parte x?

Las queries necesarias para responderlas se encuentran en el repositorio dentro de la carpeta **sql**, en el archivo **05-consultas.sql**.

10. Propuesta para datos semiestructurados, no estructurados o vectoriales.

Como se mencionó anteriormente, este proyecto cuenta con distintos tipos de datos, para su manejo se propuso:

- Encontramos datos **semiestructurados** en entidades que se benefician de cierta flexibilidad para representarlas, se dejó un campo de **JSONB** para cubrirlas. La idea es poder tener esa representatividad, sin perder la relacionabilidad ni tener que incorporar otra base de datos no relacional aparte. Ejemplos de esto se observa en los atributos
 - metadata de la tabla **archivos**.
 - metadata de la tabla **documentos**.
 - metadata de la tabla **partes**.
 - metadata de la tabla **proveedor_parte**.
- En cuanto a los datos **no estructurados**, se propuso incorporar un servicio de **minio** para almacenarlos, siguiendo un formato de carpetas que permite el versionado con el formato **{bucket}/{id}/{version}**. Luego las **URLs** correspondientes se almacenan en la base de datos relacional, lo que permite vincularlas para mantener el registro y facilitar el acceso. Ejemplos de estas son:
 - **Archivos**
 - **Documentos**
- Por último, en lo que respecta a los datos **vectoriales**, solo tenemos una entidad que los incorpora, que es la de fragmentos a través del atributo embeddings. En este caso se incorporó la extensión de **pgvector** y se creó la tabla de fragmentos que permite almacenar:
 - **id** del documento asociado
 - **contenido** de texto del fragmento
 - **vector** de embedding del fragmento
 - **modelo** de embedding utilizado
 - **número** ordinal del fragmento

Dentro de la implementación mínima se tuvo en cuenta que el borrado del documento original acarree el borrado de los embeddings asociados, lo cual permite un mejor control de los mismos. Adicionalmente, se podría haber implementado dentro de la solución un atributo que indique si está vigente o no, con el fin de establecer un trigger que se dispare al modificar algún documento y setee el flag de vigencia para poder identificar aquellos documentos cuyos embeddings han de ser actualizados. Un paso más allá podría ser también ejecutar un script para automatizar la carga de los embeddings cada vez que se crea o modifica un documento.

11. Arquitectura de datos

Este proyecto en concreto no maneja un gran volumen de datos, lo cual en cierta medida simplifica la arquitectura necesaria. Debido a que se necesita almacenar datos no estructurados, con su control de versiones y reportes, se propone utilizar un data lake. El mismo está compuesto por:

- **Bucket de archivos:** Se almacena con la estructura `/{{archivo_id}}/{{n_version}}/`
- **Bucket de documentos:** Se almacena con la estructura `/{{doc_id}}/{{n_version}}/`
- **Bucket de reportes:** Se almacenan reportes semanales y mensuales de ingresos, egresos, stocks y movimientos de stock.

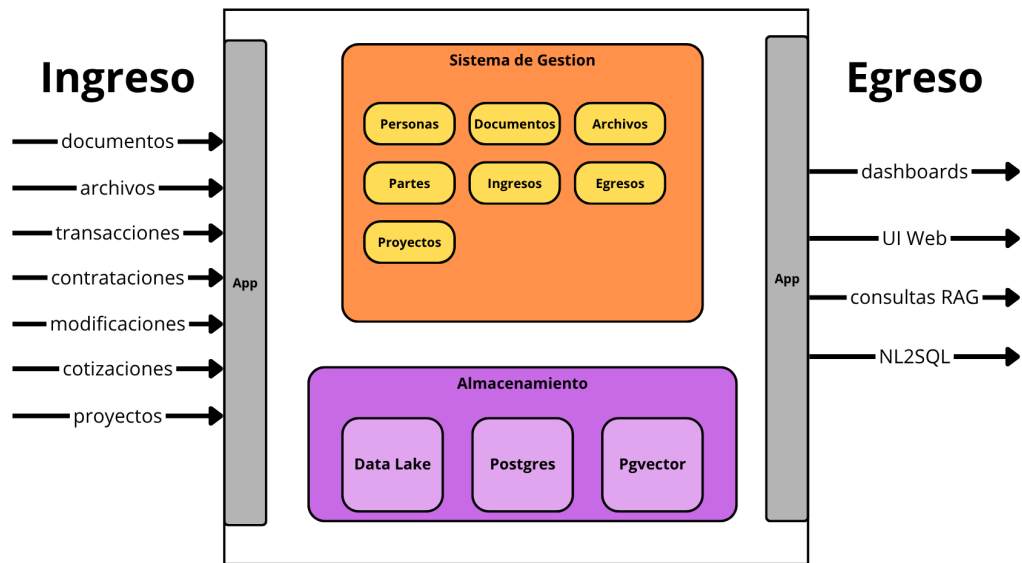
Los datos pueden ingresar al sistema a través de la capa de aplicación. Esta debe permitir:

- Creación, modificación y eliminación de proyectos.
- Creación, modificación y eliminación de archivos.
- Creación, modificación y eliminación de partes.
- Creación, modificación y eliminación de presupuesto.
- Asociaciones entre archivos y partes
- Asociaciones entre partes y proyectos
- Asociaciones entre documentos y proyectos
- Asociaciones entre proyectos y personas.
- Registro de ingresos
- Registro de egresos
- Registros de movimiento de stock

El acceso de los datos a través de la capa de aplicación permite que los datos que ingresen, lo hagan de forma limpia y esperable siempre y cuando la implementación de los endpoints sea correcta. Esto implica que si se trasladan los esfuerzos a esta parte del desarrollo, los datos del sistema en general serán limpios, siendo el mayor inconveniente el manejo erróneo de las versiones de los documentos por parte de los usuarios.

Los datos pueden ser consumidos por los usuarios a través de:

- Una **interfaz web** que permite fácilmente ver el estado de los proyectos, documentos, partes y movimientos.
- **Dashboards** generados a partir de los datos almacenados en el bucket de reportes.
- Consultas de la documentación mediante el sistema **RAG** a través de los embeddings almacenados en pgvector.
- Consultas genéricas al sistema en lenguaje natural mediante **NL2SQL**.

**Figura 3 - Arquitectura de datos**

12. Seguridad, permisos y aislamiento

La estrategia de seguridad del sistema está pensada sobre distintas capas:

- En primer lugar, a nivel de **red** se restringe únicamente el acceso de servicios que estén en la misma red local que la base de datos, en este caso **FastAPI** para implementar los endpoints de acceso y **PgAdmin** para interactuar de forma más fácil con la misma.
- Se restringe el acceso a nivel de **aplicación**, forzando la interacción del usuario a través de los endpoints definidos en la **API**.
- Se definen 3 tipos de usuarios:
 - **VIEW_ONLY**: Solo pueden ver información si mantiene relación con la misma
 - **FULL_ACCESS**: Permiso de visualización y modificación irrestricto
 - **RESTRICTED**: Permiso de visualización y modificación de información si mantienen relación con la misma
- Se protegen los documentos con 3 niveles de acceso:
 - **PUBLICO**: Cualquiera puede verlo
 - **PRIVADO**: Solo aquellos usuarios que sean empleados
 - **RESERVADO**: Sólo quienes estén vinculados o sean usuarios **FULL_ACCESS**
- Se definen 3 roles en **postgres** con diferentes niveles de acceso.
 - **Admin**: Acceso Irrestricto. La idea es utilizarlo solo por administradores que se conecten a la base de datos directamente para operaciones concretas.
 - **App**: CRUD con RLS. Es utilizado por la mayoría de los endpoints de FastAPI
 - **AI_agent**: Solo Select y RLS. Es utilizado por los endpoints de FastAPI utilizados para consultas RAG y NL2SQL.
- Se implementa **RLS** sobre:
 - **Documentos**
 - **Archivos**
 - **Empleados**
- Las consultas realizadas a través de la **API** son autenticadas mediante tokens **JWT** para identificar el id del usuario que las realiza. Esto permite además poder registrar su id en la carga de ingresos, egresos y movimiento de stock para mejor trazabilidad.

13. Escalabilidad y rendimiento

En este proyecto, el volumen de datos es bajo, sin embargo, con el tiempo es probable que haya una gran cantidad de almacenamiento dedicado a documentos y archivos. Debido a que estos son almacenados en un data lake, el escalamiento horizontal del mismo es trivial, por lo que no debería ser un problema para la infraestructura en general.

Un posible cuello de botella en el sistema puede deberse a que las políticas de **RLS** requieren hacer joins. Por ejemplo, si un usuario **VIEW_ONLY** quiere acceder a un documento, se deberá hacer una query a proyectos_documentos, para obtener los proyectos asociados y luego hacer otra query para ver las personas asociadas al proyecto y recién ahí verificar si el id del usuario está en esa lista. Como solución, se propone indexar estos ids para poder agilizar este proceso.

También se puede agilizar la búsqueda vectorial implementando indexación si esta parte del servicio se vuelve muy lenta cuando la información documental aumenta.

14. Conclusiones

Se logró esquematizar la solución de datos para el problema propuesto aplicando los conocimientos adquiridos durante la cursada. La implementación mínima permitió validar algunas decisiones de diseño, pero lo más probable es que, cuando se avance en el desarrollo y se implementen todas las funcionalidades faltantes, algunos factores deban ser revisados para hacer una aplicación limpia desde la infraestructura y que se adecue al problema de negocio y sus usuarios.